

Capítulo 3 - Camada de Transporte

↳ Um protocolo da camada de transporte fornece "comunicação lógica" entre processos de aplicação que rodam em hospedeiros diferentes. Pacotes denominados "segmentos".

Camada Transporte: Comunicação Lógica entre processos em hospedeiros diferentes

Camada Rede: Comunicação Lógica entre hospedeiros.

Sobre camada de Rede...

↳ O IP oferece comunicação lógica entre hospedeiros em um serviço de entrega de melhor esforço (não fornece garantia de entrega, ordem e integridade dos dados) → Serviço NÃO confiável

↳ Portanto, a responsabilidade fundamental do UDP e do TCP é ampliar o serviço de entrega do IP entre dois sistemas finais para um serviço de entrega entre dois processos que rodam no sistema final.

3.2 Multiplexação e Demultiplexação

Demultiplexação: Tarefa de entregar o dado contido em um segmento da camada de transporte ao socket.

Multiplexação: Trabalho de reunir, no hospedeiro de origem, porções de dados provenientes de diferentes sockets, encapsular cada porção de dados com informações de cabeçalho para criar segmentos e passar esse segmento para a camada de rede.

Cada processo usa uma porta, cada número de porta é composto por 16 bits no início de 0 a 65535. (0 a 1023 = Número de portas bem conhecidos).

No UDP o cliente possui uma porta entre (1024-65535) e o servidor uma porta entre 0 a 1023, a demultiplexação é baseada puramente na porta de destino. Porta Origem serve como endereço de retorno.

↳ Socket UDP = (IP destino, Porta Destino)

Onde ele não diferencia conexão, e vários clientes usam o mesmo socket.

No TCP o socket é: (IP origem, porta origem, IP destino, porta destino).

Onde cada cliente possui um socket diferente, mesmo que as portas de origem sejam iguais.

3.3 Transporte Não Orientado a Conexão: UDP.

O UDP faz apenas o mínimo que um protocolo de transporte pode fazer.

No receptor o UDP, a aplicação estará "falando" quase diretamente com o IP.



UDP pega a mensagem do processo da aplicação, anexa o campo de número de porta de origem e destino para o serviço de multiplexação, adiciona o campo e passa o segmento à camada de rede. **UDP = não estabelece conexão por não ser handshake.**

Porque escolher o UDP?

- Para ter melhor controle no nível da aplicação sobre quais dados são enviados e quando, quando a aplicação é tolerante a perda...
- Por não precisar estabelecer conexão (não introduz nenhum atraso a mais)
- UDP não mantém nem mantém estado de conexão e não monitora nenhum parâmetro.
↳ (um servidor dedicado pode suportar mais clientes com UDP do que TCP).
- Pequeno cabeçalho. 8 bytes X 20 bytes do TCP.
- O exemplo clássico para uso são as streams → Tolerável a perda.

3.3.1 - Estrutura do Segmento UDP.

O cabeçalho UDP possui 4 campos. (cada um de 2 bytes)

Número Porta Origem: Indica quem está enviando o pacote, varia de 0 a 65535, usado para o destinatário saber onde responder.

Número Porta Destino: Indica quem deve receber o pacote no destino, varia de 0 a 65535 necessário para saber onde deve se entregar o dado.

Comprimento: Indica o tamanho total do datagrama UDP (cabeçalho + dados), servindo para o destinatário saber onde o pacote termina. **Valor min: 8 bytes.**

Checksum: Usado para detectar erros durante a transmissão.

↳ **Cabeçalho UDP, dados e porta do IP** são verificados se o valor não está no destino o pacote é descartado.

3.3.2 - Forma de verificação UDP.

↳ Serve para detectar erros (determinar se o bits dentro segmento UDP foram alterados durante a transmissão de origem até o destino). O UDP no lado remetente realiza o complemento de 1 da soma de todos os pulsos de 16 bits do segmento, levando em conta o "vai um" em toda a soma. O resultado é colocado no campo de soma de verificação no segmento UDP.

tilibra O UDP faz checksum por nem toda o enlaces fazem verificação de

erro, e erros podem ocorrer dentro de roteadores, logo o UDP deve prover detecção de erro fim a fim na camada de transporte caso quisermos que o serviço de transporte de dados fim a fim forneça detecção de erro.

3.4 Princípios da transferência confiável de dados.

Com o protocolo TCP, nenhum dado transferido é corrompido, nem perdido, todos são entregues na ordem que foram enviados.

Caso o canal subjacente é completamente confiável. (Apenas Teórico)

Rdt 1.0 = Todo o fluxo de pacotes flui do remetente para o destinatário, com um canal perfeitamente confiável, onde não há necessidade de o lado destinatário fornecer qualquer informação ao remetente, já que nada pode dar errado.

Transferência Confiável de dados por um canal com erro de bit, rdt 2.0.

Rdt 2.0 = Nesse tipo de modelo em diante é necessário o reconhecimento positivo e o reconhecimento negativo. (Esse tipo de mensagem faz com que o destinatário saiba o que foi recebido com erro e o que foi recebido corretamente).

→ ARQ (Automatic Repeat Request) → Precisamos de 3 capacidades para manipular erro

- Detecção de Erro = É necessário um mecanismo que permita o destinatário detectar quando ocorreu erro.

- Realimentação do destinatário: O único modo do remetente saber qual é a visão do mundo do destinatário é o destinatário fornecer realimentação explícita.

→ ACK = Resposta Positiva = 1. → NAK = Resposta Negativa = 0.

- Retransmissão: Um pacote recebido com erro no destinatário para ser reenviado.

O protocolo rdt 2.0 é conhecido como protocolo pare e repare

→ Não trata a possibilidade do pacote ACK ou NAK estar corrompido.

Existem 3 possibilidades para tratar ACK/NAK corrompido.

- Perseguir novamente → (Pode causar um loop infinito de confusão) ^{e a mensagem pode sempre errar.}
- Fazer um checksum ainda mais forte, para o remetente detectar e reenviar erro de bit.

→ Resolver problema para um canal que pode corromper pacotes, mas não perdê-los. CARD é pouco eficaz

- Reenviar o pacote → Duplicação de pacotes, não é possível saber a priori se um pacote que chegou contém novos dados ou é retransmissão.

Porém a solução mais simples é: adicionar um novo campo ao pacote de dados e fazer o remetente numerar os pacotes de dados colocando um número de sequência nesse campo. O destinatário apenas verificaria se esse número é para determinar se o pacote é ou não uma retransmissão. (0 ou 1) apenas.

último recebido	chegou agora	significado
0	0	duplicado.
0	1	novo.
1	1	novo.
1	0	duplicado.

RTT 2.0 = Cada rdt 2.0 remetente e destinatário possui duas vezes + estado.

Destinatário

- Esperando Pacote 0: ^{pacote}chega 0 → aceita, envia ACK e espera 1, chega 1 → ignora mesmo ACK antigo.
- Esperando Pacote 1: → Mesmo coisa porém recebe primeiro.

Remetente

- Enviando 0: recebe ACK → envia 1. ^{→ NAK} erro envia 0.
- Enviando 1: recebe ACK → envia 0. erro envia 1.

RTT 2.2 = Mantém a mesma lógica do Rdt 2.1, porém substitui NAK por ACK duplicado. (Mantém lógica mais simples, menos chance de erro, mesmo efeito).

→ Enviando 0: recebe ACK 0 → envia 1 recebe ACK 1 → reenvia 0.

→ Enviando 1: recebe ACK 1 → envia 0 recebe ACK 0 → reenvia 1.

~~RTT 2.2 transforma NAK explícito em erro implícito via ACK duplicado.~~

~~RTT 2.2 transforma NAK explícito em erro implícito via ACK duplicado.~~

Transferência confiável de dados por um canal com perda e com erros de bits. RDT 3.0

Nesse caso, precisamos saber como detectar perda de pacote e o que fazer diante dela.

→ Atribuímos ao remetente o ônus de detectar e se recuperar das perdas de pacotes.

A técnica adotada é: o remetente faz uma escolha ponderada de um valor de tempo dentro do qual seria provável, mas não garantido, que a perda tivesse ocorrido, se o ACK não foi recebido no período, o pacote é retransmitido.

→ Como o pacote não tem a perda, mas atrasado cria o problema de pacotes

tilibra Duplicados entretanto, o RDT 2.2 já trata.

Para implementar o mecanismo de retransmissão baseado no tempo é necessário temporizador de contagem regressiva. Sendo preciso o remetente acionar o temporizador toda a vez que um pacote for enviado, responder uma interrupção feita pelo temporizador e parar o temporizador.

Bit 3.0 implementa esse mecanismo citado acima e possui um nome característico → Protocolo do Bit Alternante. Porque esse bit fica alternando entre 0 e 1 a cada pacote enviado corretamente. (Stop-and-Wait) / pack por vez.

Protocolo de transferência confiável de dados com paralelismo.

O rdt 3.0 funciona perfeitamente entretanto o desempenho dele hoje é **LENTO**.

↳ Stop-and-Wait dependendo muito quando o RTT é grande.

Dai surge o paralelismo, se for possível enviar mais pacotes por vez, o envio é ^{mais} rápido.

↳ Porém isso traz consequências...

- O que eram apenas 0/1, são ^{agora} ~~pacotes~~ ^{pacotes} em trânsito (cada ^{um} precisa de número único)
- Precisa de buffer: remetente precisa guardar pacotes enviados mas não confirmados.
- destinatário → pode precisar guardar pacotes fora de ordem.
- A faixa de número de sequência necessário dependerá de como um protocolo de transferência de dados responde a pacotes perdidos, corrompidos e duplicadamente recebidos.

3.4.3 GO-BACK-N

↳ O remetente é autorizado a transmitir múltiplos pacotes sem esperar por um reconhecimento, porém fica limitado a ter não mais do que algum núm max permitido de pacotes na "tubulação".

"base" = número de sequência do mais antigo pacote não reconhecido.

"nextseqnum" = menor número de sequência não utilizado.

Número $[0, base-1]$ = Pacotes que já foram transmitidos e reconhecidos. ^{Já confirmado.}

$[base, nextseqnum-1]$ = pacotes enviados, mas que ainda não foram reconhecidos. ^{aguardando ACK.}

$[nextseqnum, base+N-1]$ = pacotes que podem ser enviados imediatamente, caso cheguem dados vindo de camada superior. ^{pode enviar.}

Num Seq $> base+N$ = Não podem ser usados até que um pack não reconhecido que esteja pendente seja reconhecido. ^{Não pode enviar.}

Janela: Faixa de números de sequências permitidas para parâmetros transmitidos, porém ainda não reconhecidos onde N é o tamanho da janela.

Go Back N = Protocolo de Janela Deslizante.

9 normalmente de GO BACK N debe responder 3 eventos.

- **Chamada vindida de cima:** quando `rdt_send()` é chamado de cima o remetente verifica se a janela está cheia, se não, o pacote é criado e enviado e as variáveis ^{no} `sequence` e `window` de `rdt` são atualizadas. De `rdt` para cima, o remetente apenas desloca o dado (ou quando num buffer).
- **Recebimento de um ACK:** reconhecimento de pacote com número de sequência n que é tratado como um reconhecimento cumulativo, o que indica que todos os pacotes até N também são corretamente recebidos no destinatário.
- **Timeout:** **UM** temporizador é usado para recuperar a perda de dados ou reconhecer pacotes. De ocorrer esgotamento da temporização, o remetente reenvia todos os pacotes que tinham sido previamente enviados, mas que ainda não tinham sido recebidos. De um ACK ser recebido, mas ainda houver pacotes adicionais transmitidos mas não recebidos, o temporizador será reiniciado. Se não houver pacote pendente não ^{reenvia} e o temporizador será desligado.

Se um pack com número de sequência n for recebido corretamente e estiver na ordem, o destinatário enviará um ACK para o parete n e entregará a parte de dados do pack à camada superior. **Por outro lado,** o destinatário descarta o pacote e retransmite um ACK para o mais recente que foi recebido na ordem correta, se o pack K foi recebido e entregue, todas as outras partes com número de sequência menor que K também foram entregues.

9 GO BACK N desce até a palavra que chegou ao fim de ordem, se a palavra não foi encontrada, tenta outras palavras resultando das regras de reestruturação do GB1

↳ Vantagem é a complexidade da manipulação de soffere no dominicano

↳ Construção é ter a chance de retransmissão também por erro.

Repetição Seletiva (SR)

O grande defeito do protocolo GO BACK N, é o fato de que quando um pacote é perdido, ele retransmite todo (desperdiça banda, resolve retransmitindo até, pagar um pouco).

tilibra 9 SR operar retransmite o pacote que deu erro.

De um modo geral, o SR funciona da seguinte forma:

- ↳ O remetente pode enviar vários pacotes (varia de 1 a N)
- ↳ Cada pacote tem seu próprio temporizador. ↳ Recebe ACKs individuais.
- ↳ Destinatário: aceita os pacotes fora de ordem, guarda em um buffer e a entrega à aplicação só é feita EM ORDEM.

O problema do SR é quando o TAM-JANELA = ESPAÇO-SEQUENCIA porque em caso assim, podem ter ambiguidade pro destinatário, pois assim ele não sabe se é retransmissão ou se é um pacote novo.

Regra fundamental SR

$$\text{Tam-janela} \leq (\text{Espaço sequência}) / 2$$

3.5 Transporte Orientado Para Conexão: TCP → (Ponto a Ponto)

O TCP é orientado para conexão porque, ele primeiro precisa se "apresentar" antes de começar a enviar dados a outro. A conexão TCP trata-se de uma "conexão lógica" com o estado comum residindo apenas na TCPs no dois sistemas ligados em comunicação.

- TCP possui uma conexão conexão full duplex. Os dados da conexão de aplicativos podem fluir de A para B ao mesmo tempo que pode de B para A.

Processo Cliente: Processo que está iniciando conexão, o outro Processo Servidor.

O papel do buffer dentro do TCP.

- ↳ Buffer de Envio: Guarda tudo antes de enviar, TCP decide quando enviar e pode juntar vários dados em um segmento. (TCP não envia imediatamente tudo que recebe)
- ↳ Buffer de Recepção: Guarda dados recebidos, podendo receber fora de ordem porém entrega à aplicação em ordem.

TCP não manda tudo de uma vez, ele divide: MTU unidade máxima de transmissão.

3.5.2 Estrutura Segmento TCP

Cabeçalho TCP possui geralmente 20 bytes, 12 a mais que o cabeçalho UDP.

- Possui número de origem e de destino. 16 bits cada
- Possui um campo de Checksum. 16 bits
- Campo de número de sequência e campo número de reconhecimento que não serão usados pelo

TCP remetente e destinatário na execução de um serviço conjunto de transferência de dados.

- Janela de recepção é usada para controle de fluxo e é usada para indicar o número de bytes que um destinatário está disposto a aceitar 16 bits.

tilibra

• Campo do comprimento de cabeçalho especifica o comprimento do cabeçalho TCP em palavras de 32 bits **4 bits**

• Campo de opções, é usado quando remetente e destinatário negociam o MSS. **Variable 0-60**

• Campo Flag possui **6 bits**, BIT ACK = usado para saber se o segmento contém um reconhecimento para um segmento que foi recebido com sucesso. RST, SYN FIN usada para estabelecer e encerrar conexões. CWR e ECE = notificação explícita de congestionamento. PSH marcado indica que o destinatário deve passar dados a camada superior, URG usado para mostrar que há dados nesse segmento que camada superior precisa com urgência.

No TCP, as numerações não são por pacote mas sim por byte.

Arquivo 5000 bytes MSS=1000 bytes Seq, 1 = 0-999, Seq, 2 = 1000-1999 ... Seq, 5 = 4000-4999.

→ número de sequência para um segmento.

Número de Reconhecimento: O Hóspedeiro A atribui a seu segmento é o número de sequência do próximo byte que ele estiver aguardando do Hóspedeiro B.

Por exemplo com pacote 1500 bytes e MSS 500 bytes **ACK cumulativo.**

1 [0-499] - Enviado com sucesso, envia ack 500.

2 [500-999] - Enviado com sucesso, ack 500

3 [1000-1499] - Enviado com sucesso, envia ack 500

TCP só confirmou até o primeiro sucesso.

→ O TCP guarda num buffer o pacote 3, mas espera o recebimento do pacote 2 ou seja ACK 500, qual o pacote 2 é recebido ele manda o ack 1500.

O TCP detecta perda comparando o número de sequência recebido com o próximo byte esperando, sem depender de ACKs para identificar lacunas.

3.5.3 - Estimativa do tempo de viagem de ida e volta e de resgate de tempo-rigidez.

Quando o TCP envia um segmento, ele espera um ACK. Porém, ele precisa saber se o atraso é um atraso ou se o ACK se perdeu.

→ Esse tempo de ida e volta é chamado **Round Trip Time**.

Sample RTT = RTT de um segmento específico.

→ Nunca é computado para um segmento que foi retransmitido.

Para calcular o Sample o RTT mantém uma média - **Estimated RTT**.

tilibra
$$\text{Estimated RTT} = (1 - \alpha) \cdot \text{Estimated RTT} + \alpha \cdot \text{Sample RTT}$$

$\alpha = 0,125 \rightarrow$ valor recomendado.

Como RTT é uma estimativa do atraso entre Sample RTT e Estimated RTT.

$$\text{Dev RTT} = (1-B) \cdot \text{Dev RTT} + B \cdot |\text{Sample RTT} - \text{Estimated RTT}| \quad B = 0,25 \text{ recomendado.}$$

Portanto, a recomendação de intervalo é

$$\text{TimeoutInterval} = \text{Estimated RTT} + 4 \times \text{Dev RTT}$$

3.5.4 Transferência confiável de dados.

Tendo em vista que o IP não é confiável (por poder chegar fora de ordem, corrompido)

o TCP cria um serviço de transferência confiável de dados sobre o serviço de melhor esforço do IP. *Dequem - as maneiras*

- **Duplicação do tempo de expiração:** Nessa modificação, sempre que ocorre algum erro, o TCP retransmite o segmento ainda não recebido que tenha o menor número de sequência. Mas a cada retransmissão, o TCP ajusta o próximo tempo de expiração para o dobro do valor anterior em vez de derivá-lo da última Estimated RTT e Dev RTT.

→ **Forma limitada de controle de congestionamento.**

- **Retransmissão rápida:** Basicamente, já sabemos que o TCP usa a técnica de ACKs duplicados, → para indicar que ainda está esperando um segmento. A retransmissão rápida, portanto, apenas age retransmitindo o segmento faltante, ~~sem~~ sem esperar o timeout assim que se acumulam 3 ACKs. → Forte evidência de perda.

1 ou 2 ACKs = Podem ser apenas reordenação na rede

o TCP = mais próximo do GBN, mas corrigido com SACK.

o TCP = muito próximo do SR, com SACK.

→ o TCP é um híbrido entre GO BACK N e SELECTIVE REPEAT.

3.5.5 Controle de Fluxo.

- o TCP garante que o remetente não envie mais dados do que o receptor consegue armazenar.

→ o receptor (B): calcula rwnd, envia esse valor no campo de janela do TCP.

→ o remetente (A): controla quanto enviou e não foi ACKado. → Espaço Livre no Buffer.

→ Last Byte Sent - Last Byte Acked garante Last Byte Sent - Last Byte Acked < rwnd

→ Basicamente, nunca envia mais do que o receptor aguenta.

Caso B = "rwnd = 0" → A para de enviar **PROBLEMA**

o TCP resolve ao ^Acontinuar enviando pequenos segmentos (1 byte) periodicamente

B responde com ACK, quando houver espaço rwnd > 0

A volta a transmitir normalmente.

tilibra

3.5.6 Estabelecimento da conexão TCP.

0 TCP vai a chamada "apresentação de nós vizinhos"

1 Cliente → Servidor (SYN) "Quero iniciar conexão"

É enviado um segmento TCP especial pro lado servidor do TCP, onde o bit flag SYN é ajustado para um, e o número de sequencial inicial é (cliente.ism).

2 Servidor → Cliente (SYN + ACK) "Recebi, vamos conectar. Meu número é este."

O servidor extrai o segmento TCP SYN da datagrama, coloca no buffer e verifica o TCP. A conexão é enviada um segmento de aceitação de conexão ao TCP cliente. Onde esse segmento, o bit SYN está com valor 1, o campo de reconhecimento do segmento TCP está ajustado para $client.ism + 1$, e o servidor escolhe seu próprio número de sequência inicial ^{num} _{sm}.

3 Cliente → Servidor (ACK) "Conexão Estabelecida"

Após receber o segmento SYNACK, o cliente também remove o buffer e verifica o segmento.

O lado cliente envia ao servidor mais um segmento. Este reconhece o segmento de confirmação da conexão do servidor. O bit SYN é ajustado para 0, pois a conexão foi estabelecida.

Existem 3 etapas para garantir que: ambos estão prontos, números de sequência estão sincronizados, não é uma conexão fantasma.

Encerramento de Conexão

Se uma aplicação cliente decide que quer fechar conexão.

→ O TCP cliente envia um segmento TCP com o bit FIN em 1 e entra no estado FIN-WAIT_1. No estado FIN-WAIT_1, o TCP cliente espera por um segmento TCP do servidor com um reconhecimento.

Após receber esse segmento, o TCP cliente entra no estado FIN-WAIT_2. No estado FIN-WAIT_2, ele espera por outro segmento do servidor com o bit FIN ajustado para 1. O TCP cliente reconhece o segmento do servidor e entra no estado TIME-WAIT.

Este estado permite que o TCP cliente receba o reconhecimento final, caso o ACK seja perdido. O tempo passado no estado TIME-WAIT depende da implementação, mas os valores típicos são 30 segundos, 1 minuto e 2 minutos. Após a espera, a conexão se encerra formalmente e todos os recursos do lado cliente.

3.6 Princípios de controle de congestionamento.

Causas e os custos do congestionamento.

O congestionamento ocorre quando vários computadores tentam usar a rede ao mesmo tempo, se essa rede não suporta o tráfego, existe acúmulo de fila/perda de pacotes.

1º Cenário: Basicamente, nessa parte o que ocorre é que se R = capacidade max enlace, porém dois hospedeiros transmitindo dados há $R/2$ (mesmo que não ocorra perda de dados), ainda o roteador recebe dados muito rápido quanto consegue enviar. (Portanto mesmo que contraditório, enviar dados a uma taxa próxima a $R/2$ pode ser ruim por gerar muitos atrasos).

2º Cenário: Em uma rede congestionada - o remetente deve realizar retroalimentação para compensar o pacote descartado pelo esgotamento do buffer. Ainda tendo mais um custo, que não as retransmissões desnecessárias feitas pelo remetente em face de grandes atrasos podem fazer um roteador usar sua largura de banda de enlace para retransmitir cópias desnecessárias de um pacote.

3º Cenário: O descarte de pacotes devido ao congestionamento - quando um pacote é descartado ao longo de um caminho, a capacidade de transmissão que foi usada em cada um dos enlaces anteriores para retransmitir o pacote até o ponto que foi descartado acaba sendo desperdiçada.

Mecanismos de controle de congestionamento

Controle Congestionamento fim a fim: Basicamente, nesse tipo de controle a rede não ajuda, não envia avisos e não informa nada. O TCP que percebe o congestionamento (por perdas de pacotes ou aumento dos atrasos) → 3ack duplicado, pack enviado mais rápido.

O TCP basicamente reduz a janela e aumenta o tempo RTT.

Controle Congestionamento assistido pela rede: Os roteadores fornecem retroalimentações específicas de informações ao remetente a respeito do congestionamento da rede. (Feedback direto / Marcações de pacotes).

3.7 Controle de congestionamento no TCP.

O TCP usa controle de congestionamento fim a fim em vez de controle assistido pela rede já que a camada IP não fornece os mecanismos finais realmente específicos relativos ao congestionamento da rede.

O TCP obriga cada remetente a limitar a taxa pela qual enviam tráfego para sua conexão como uma função do congestionamento percebido. (De um remetente perceber ponto congestionamento no caminho → aumenta taxa de envio, se há congestionamento → reduz.

O TCP limita sua taxa de envio através de uma janela de congestionamento, **cwnd**.

cwnd: Define quantos bytes podem estar na "rede" sem ACK.

Last Byte Sent - Last Byte Acked $\leq$ cwnd

Portanto, o TCP controla velocidade ajustando cwnd

A taxa de envio do remetente é aproximadamente $cwnd / RTT$ bytes por segundo. Portanto, ajustando o valor de cwnd, o remetente pode ajustar a taxa pela qual envia dados para a sua conexão.

Como o TCP percebe congestionamento? Perda de pacote, ACK demorou muito (timeout), ^{3 acks duplicados}

Como o TCP ajusta sua taxa? **Segmento perdido** $\rightarrow$ diminuir cwnd

ACK recebido $\rightarrow$ aumentar cwnd

O TCP faz isso o tempo todo: aumenta taxa, observa rede, quando ocorre perda: reduz taxa, e depois volta a aumentar de novo. **TCP é autorregulado**; ACKs ^{controlam o ritmo novo.}

O TCP não conhece a capacidade da rede, não conhece largura de banda, não conhece buffer

$\rightarrow$ Aprende dinamicamente.

Partida Lenta (Algoritmo de controle de congestionamento TCP)

Nele o "cwnd" = MSS, onde ele vai crescer exponencialmente

$\rightarrow 1, 2, 4, 8, 16, 32, \dots$

É a partida lenta irá continuar crescendo exponencialmente assim até ocorrer algum dos três problemas: **cwnd alcançar ssthresh**, (a partir desse limite começa o crescimento linear...), **timeout** (o sender envia metade do cwnd (onde ocorreu time-out))

(ex = cwnd = 20, ssthresh = 10), a partir daí o cwnd volta para 1, e cresce exponencialmente até chegar no limite ssthresh, a partir do limite cresce de 1 em 1, ^{3 acks du-}

Perda (congestionamento não tão grave: pacote continua chegando) o TCP detecta a perda do pacote que faltou, retransmite ele imediatamente, reduz ssthresh pela metade e o **cwnd = ssthresh + 3MSS** e assim volta para crescimento linear (+1)

* O caso de 3 acks duplicados é o chamado fast retransmit.

TCP Tahoe = Qualquer perda cwnd = 1. **TCP Reno**: usa fast retransmit

TCP Cubic: O CUBIC muda a par de presença de congestionamento

$\rightarrow$ Ele basicamente volta rapidamente a última janela antes da perda.

$20 \rightarrow 10 \rightarrow 15 \rightarrow 18 \rightarrow 20 \rightarrow 20.2 \rightarrow 20.4 \dots$ CUBIC $W(t) = C(t-k)^3 + W_{max}$.

tilibra $20 \rightarrow 10 + 11 + 12 \dots 20 - 21$ **RENO**.

ECN - Explicit Congestion Notification

↳ No TCP clássico, o congestionamento só é percebido depois que deu problema.

Quando o buffer do roteador começa a encher, ao invés dele descartar pacotes ele marca bits ECN no cabeçalho IP. O receptor vê que o pacote está marcado e coloca o bit ECE no ACK. Ao receber esse ACK com ECE 1, "a rede está congestionada" e reage igual um fast retransmit "reduz cwnd pela metade".

Controle baseado em atraso (TCP Vegas): ^{para evitar} "De o RTT está crescendo, provavelmente encheando o buffer, portanto mede o RTT min (atraso ideal), onde o segm ficou comprando o tempo, todo se o RTT atual é $\geq$ o mínimo caso for = mesmo tamanho CWND aumenta, se for muito maior CWND diminui".

O TCP tenta dividir a largura igualmente entre conexões concurrentes. De um enlace para capacidade R e existem K conexões TCP cada conexão tende a receber aproximadamente R/K . Isso ocorre por causa do algoritmo de AIMD: aumenta cwnd lentamente (1 srtt por RTT) e quando há perda, reduz pela cwnd pela metade. Assim, conexões com taxas diferentes tendem a convergir para valores semelhantes (padrão "dente de serra").

Cubic: A new friendly-TCP High Speed TCP Variant

Existem diversas variantes do TCP que procuram resolver o problema da subutilização da rede, causando principalmente pelo crescimento lento da janela de congestionamento do TCP em rede com grande produto banda-atraso. Elas modificam a função de crescimento da janela de congestionamento para torná-la mais escalável, porém precisam resolver o problema de justiça.

TCP-friendliness: determina se um protocolo é justo em relação ao TCP tradicional.

↳ Característica crítica para a segurança da adoção de um novo protocolo, pois seu uso não prejudica injustamente o tráfego TCP convencional.

• Cenário com alta taxa de perda, onde o TCP apresenta um comportamento adequado o novo protocolo deve se comportar de maneira semelhante ao TCP.

• Cenário com baixa taxa de perda, o TCP apresenta problema de utilização da rede, o novo protocolo pode utilizar mais largura de banda que o TCP.

A maioria das soluções usam um "modo TCP": "Se a janela está pequena, comporta-se como TCP".

O problema é que isso usa apenas o tamanho da janela para decidir quando agir como TCP.

Os autores afirmam que "O comportamento do TCP não depende apenas da taxa de perda, ele depende fortemente do RTT, → a mesma janela pode representar situações completamente diferentes a depender do RTT".

CUBIC

↳ Manter a escalabilidade do BIC, manter sua estabilidade, melhorar a justiça, melhorar a compatibilidade com TCP.

• O crescimento da janela passa a depender do tempo transcorrido desde a última perda, assim o protocolo consegue operar de modo semelhante ao TCP quando a taxa de perda é alta ou quando o RTT é pequeno.

BIC

↳ Principal característica: função utilizada para aumentar janela de congestionamento

↳ Ao ocorrer uma perda de pacote, o BIC reduz sua janela por um fator multiplicativo β .

Wmax: tamanho janela imediatamente antes da perda.

Wmin: tamanho janela após a redução.

porque houve perda

porque não houve perda.

A capacidade real da rede deve estar entre Wmax e Wmin.

Assim, se a distância até o ponto médio for maior que uma constante S_{max} , o BIC aumenta a janela apenas em S_{max} (crescimento linear).

Após esse aumento, o novo valor torna-se o novo mínimo e se houver perda, o novo valor se torna o máximo. → Protocolo considera boa estimativa da capacidade rede.

Esse processo continua até que o crescimento seja menor que uma constante. Sem a janela poder ultrapassar W_{max} sem gerar perdas, significa que a capacidade real da rede é maior que a estimada → nova fase → Max Backing.

Nessa fase, o crescimento inicialmente é lento, o protocolo procura uma nova capacidade máxima próxima do valor atual, se nenhuma perda for observada, ele passa a crescer cada vez mais rápido.

Entretanto, para funções de crescimento ainda pode ser muito agressivo para o TCP tradicional, principalmente em rede com RTT pequenos ou velocidade menor.

CUBIC

Janela é calculada: $W_{cubic} = C(t-K)^3 + W_{max}$.

C = fator escalonador. t = tempo transcorrido desde a última redução da janela.

W_{max} = tamanho da janela imediatamente antes da última perda.

K = parâmetro calculado a partir de W_{max} , β e C .

Logo após uma perda, a janela cresce rapidamente e ao se aproximar de W_{max} , o crescimento desacelera. Próximo de W_{max} o crescimento praticamente para e ao

ultrapassar W_{max} , o protocolo procura largura de banda adicional, começa desviar, acelera gradualmente.

- Estabilidade próxima do ponto de equilíbrio. Escalabilidade quando existe banda livre.
- Função cúbica favorece convergência para uma divisão justa de banda, o que faz que dois fluxos CUBIC competindo pelo mesmo enlace, ambos tendam a convergir para valores semelhantes de janela e largura de banda.
- A função depende principalmente do tempo transcorrido (t), como o tempo é igual para todos, independentemente do RTT, isso melhora significativamente a RTT fairness em comparação com TCP e BIC.

O crescimento baseado em tempo real melhora muito a compatibilidade com TCP, por isso, em rede com RTT pequenos, o TCP tradicional pode crescer mais rapidamente que o CUBIC, tornando o CUBIC mais amigável ao TCP.

BIC e CUBIC conseguem manter época de congestionamento longa, sem perdas.

tilibra escalabilidade, sem reduzir a utilização da rede.

- CUBIC é mais amigável ao TCP do que BIC, HSTCP e STCP.
- CUBIC mantém boa compatibilidade tanto em RTTos pequenos quanto grandes.
- CUBIC consegue combinar alta utilização da rede com longas épocas de congestionamento.
- CUBIC apresenta melhor estabilidade de estabilidade que STCP e HSTCP.
- CUBIC mantém escalabilidade sem sacrificar fairness.

Binary Increase Congestion Control for Fast, Long distance networks.

Um protocolo não deve reduzir largura de banda excessivamente do fluxo TCP tradicional, ao mesmo tempo em que consegue utilizar plenamente a capacidade da rede de alta velocidade. - **Impunção em relação ao RTT**: situações em que fluxos com diferentes RTTs podem consumir parcelas extremamente desiguais da largura de banda disponível.

O TCP é o protocolo de rede mais utilizado da rede, ele subutiliza significativamente a largura de banda disponível em conexão de alta velocidade.

Algumas soluções propõem escapar de serem escaláveis e relativamente compatíveis com

o TCP, o protocolo HSTCP e STCP apresentam grave problema de impunção de RTT.

↳ Como protocolo aumentam a janela mais rapidamente quando ela já é grande, o fluxo com RTT menor consegue crescer cada vez mais rápido do que um o fluxo com RTT maior.

BI-TCP (Binary Increase TCP)

↳ Objetivo é alcançar escalabilidade, não prejudicar excessivamente fluxos TCP tradicionais e evitar fluxos com RTT menor monopolizarem a banda.

↳ Usando: Aditivo Increase (crescimento linear da janela), Binary Search Increase (crescimento baseado em busca binária)

Janela Grande

↳ Crescimento aditivo por manter melhor justiça de RTT

Janela pequena

↳ Busca binária, por manter compatibilidade com TCP.

Realizando experimentos em um ambiente tentando espelhar a realidade, dá para perceber que muitos fluxos estarão perdendo pacotes ao mesmo tempo

↳ **Synchronized Packet Loss**: Basicamente, quando o buffer ^{do roteador} acaba muitos fluxos sofrem perda de pacotes simultaneamente.

Protocolos de alta velocidade como HSTCP e STCP conseguem utilizar melhor rede rápida, mas apresentam um sério problema de justiça: conexão com RTT menor acabam recebendo muito mais largura de banda, principalmente quando ocorrem perdas sincronizadas de pacotes em roteadores drop tail.

Função de resposta: mostra qual velocidade um protocolo consegue atingir para uma determinada taxa de perda onde cada protocolo possui uma "curva de comportamento", onde essa curva permite analisar: escalabilidade, justiça entre RTTs, compatibilidade com TCP.

Ideia principal do BI-TCP: tratar o controle de congestionamento como um problema de busca

↳ Definindo o mínimo seguro e o máximo permitido, o BI-TCP testa e ajusta médias de janelas $\uparrow$, se der perda $\downarrow$, ... depois reduzindo...

↳ Isso é interessante porque dessa maneira se chega mais rápido ao limite da rede

• O problema que poderia existir é o fato que saltos grandes: podem causar muita perda, pode prejudicar outros fluxos.

↳ **Additive Increase:** Que é basicamente crescer de maneira linear: 100-110-120...

Entanto, quando está longe do limite, ele cresce rapidamente (com o objetivo de descobrir a capacidade da rede) $\rightarrow$ "Binary Search"

Quando está perto do limite, ele decorelora (com o objetivo de ~~o~~ evitar congestionamentos) $\rightarrow$ Additive Increase

Fast Convergence: Quando um fluxo percebe que estava usando banda demais: ele reduz sua estimativa máxima (isso ajuda o fluxo a chegar mais rápido)

O BI-TCP em "Janelas Pequenas" comporta-se de forma mais parecida com TCP (for compatibilidade)

em "Janelas Grandes" ele se comporta de forma mais parecida com AIMD (melhorar utilização)

Com drop tail o BI-TCP não perde eficiência, ele consegue utilizar praticamente toda a capacidade da rede, o BI-TCP reduz significativamente o problema da RTT unfairness

O BI-TCP reduz significativamente o problema de RTT unfairness

O BI-TCP usa mais banda que o TCP entretanto ele normalmente usa a banda que o TCP não conseguiria aproveitar

↳ BI-TCP não reduz drasticamente a paralela do TCP, em vez disso ocupa espaço que outros protocolos. "Continua sendo relativamente amigável ao TCP"

Quando vários fluxos competem entre si, o BI-TCP mantém uma divisão mais justa da largura de banda.

O BI-TCP encontra mais rapidamente uma divisão justa de rede.

Conclusão

RTT - Fatorado é um requisito importante para protocolos de alta velocidade.

O BI-TCP que é um protocolo que consegue equilibrar: escalabilidade, compatibilidade com TCP, redução de RTT.

↳ Quando a taxa de perda fica extremamente baixa (menor que 10^{-3}) outros protocolos conseguem crescer mais rapidamente.

↳ Porém isso não é um grave problema pois quase não existe congestionamento e a utilização passa a depender mais da redução após a perda do que do crescimento da janela.